

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Национальный исследовательский университет
НИТУ МИСИС**

КУРСОВАЯ РАБОТА

по дисциплине «Дискретная математика»

**Тема: «Построение элементарных и составных кубических
кривых Безье»**

Выполнили студенты группы БПМ-24-4:

Сычев Арсений

Щукин Егор Ка-

лашников Егор

Еманов Алексей

Татаринов Дмитрий

Курчиев Виктор

Проверил преподаватель:

Ширкин С.В.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	3
2. АНАЛИТИЧЕСКИЙ ОБЗОР ЛИТЕРАТУРЫ	4
3. МАТЕМАТИЧЕСКОЕ ОПИСАНИЕ КРИВЫХ БЕЗЬЕ	6
3.1. Определение кубической кривой Безье	6
3.2. Коэффициентная форма уравнений	6
3.3. Геометрические свойства и условия стыковки	7
3.4. Масштабирующий алгоритм преобразования	7
3.5. Формулы деления элементарной кривой	8
4. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ	9
4.1. Построение элементарной кубической кривой (Задание 3.1)	9
4.2. Натяжение составной кривой на контур пламени (Задание 3.2)	10
4.3. Результаты масштабирования составного контура	11
5. ЗАКЛЮЧЕНИЕ	14
6. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	15
7. ПРИЛОЖЕНИЕ. ИСХОДНЫЙ КОД ПРОГРАММЫ	16

1. ВВЕДЕНИЕ

Кривые Безье представляют собой один из наиболее гибких и востребованных инструментов в современной компьютерной графике, системах автоматизированного проектирования (САПР) и шрифтовом дизайне. Главное достоинство таких кривых заключается в аналитическом (параметрическом) способе задания геометрии, что позволяет описывать сложные гладкие линии минимальным набором управляющих параметров — координат контрольных точек.

Данная курсовая работа посвящена детальному изучению, программной реализации и анализу элементарных и составных кубических кривых Безье в рамках выполнения практических заданий 3.1 и 3.2. Векторное представление графических объектов позволяет осуществлять их трансформацию (в частности, масштабирование) без потери качества и геометрической точности, в отличие от растрового подхода. Изучение данного математического аппарата позволяет связать теоретические основы дискретной математики и вычислительной геометрии с практическими задачами визуализации данных.

Целью работы является разработка программного комплекса для генерации элементарной кубической кривой Безье по заданным контрольным точкам, построения замкнутого составного контура сложной формы («Пламя»), а также реализации алгоритма его аффинного масштабирования относительно произвольного центра.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Изучить математический аппарат многочленов Бернштейна и параметрических уравнений кубических кривых Безье.
2. Реализовать перевод параметрической формы кривой в коэффициентную (полиномиальную) форму [4] и программно рассчитать коэффициенты для заданного набора точек.
3. Разработать алгоритм объединения нескольких изолированных кубических сегментов в единый непрерывный составной контур с проверкой условий стыковки точек.

4. Сформировать геометрическую модель сложного замкнутого контура («Пламя») из шести состыкованных кубических сегментов.
5. Реализовать алгоритм масштабирования полученной векторной фигуры относительно фиксированного центра координат для коэффициентов $k \in \{0.5, 1.0, 1.5, 2.0\}$.
6. Проанализировать поведение графических объектов при трансформациях и оформить результаты в виде итогового отчета.

Выбор данной темы обусловлен глубоким интересом авторов к технологиям компьютерной графики, геометрического моделирования и визуализации данных. Изучение математического аппарата и программная реализация алгоритмов построения кривых Безье представляет собой важный шаг для понимания принципов работы современных систем рендеринга, разработки векторных графических редакторов и создания вычислительных моделей.

2. АНАЛИТИЧЕСКИЙ ОБЗОР ЛИТЕРАТУРЫ

Свое название кривые Безье получили в честь выдающегося французского инженера и математика Пьера Безье [1] (Pierre Bézier), который в 1962 году, работая в автомобилестроительной компании Renault, стал успешно применять их для математического описания и проектирования кузовов автомобилей. Практически одновременно и независимо от него аналогичные исследования проводил Поль де Кастельжо (Paul de Casteljau) в конкурирующей компании Citroën. Де Кастельжо разработал численный алгоритм для вычисления точек кривой, однако из-за строгой политики конфиденциальности метод получил имя Безье.

Несмотря на западную историю внедрения в инженерную практику, фундаментальный математический аппарат, лежащий в основе этих кривых, был разработан великим русским и советским математиком академиком С. Н. Бернштейн [2]ом в 1912 году. В своих трудах Бернштейн решал чисто теоретическую задачу конструктивного доказательства теоремы Вейерштрасса (1885 г.).

Теорема Вейерштрасса [3] утверждает:

«Дана функция $f(x)$, непрерывная в замкнутом интервале $[a, b]$. Тогда для любой заранее заданной границы точности ε существует такой многочлен $P(x)$, что $|f(x) - P(x)| < \varepsilon$ во всем интервале $[a, b]$.»

До Бернштейна существовали лишь абстрактные доказательства этой теоремы. С. Н. Бернштейн [2] предложил явную конструкцию таких многочленов, используя математический аппарат теории вероятностей (схему независимых испытаний Бернулли). Выражение для многочлена Бернштейна n -й степени для функции $f(x)$ на отрезке $[0, 1]$ имеет вид:

$$B_n(f; x) = \sum_{m=0..n} f(m/n) \times C_n^m \times x^m \times (1 - x)^{n-m}$$

В компьютерной графике для построения плоских и пространственных линий используется векторный аналог полиномов Бернштейна, где вместо значений

функции подставляются координаты контролирующих точек P_i .

Современная геометрическая мозаика компьютерной графики полностью опирается на кубический случай ($n=3$), который обеспечивает оптимальный баланс между вычислительной сложностью и геометрической гибкостью.

Достоинства использования кривых Безье:

- *Компактность хранения данных.* Для описания сложного графического контура не требуется сохранять координаты тысяч растровых пикселей. Достаточно закодировать небольшой массив контрольных точек.
- *Абсолютное бесконечное масштабирование.* Объект может быть увеличен или уменьшен в любое число раз без появления дискретности или размытия контуров. Качество ограничивается исключительно разрешающей способностью выводного устройства.
- *Интуитивное интерактивное управление.* Перемещение управляющей точки плавно «притягивает» к себе соответствующий участок кривой, действуя как виртуальный магнит.

Недостатки и ограничения:

- *Программная зависимость.* Векторные изображения требуют интерпретации математическим движком, в связи с чем при конвертации между форматами могут возникать погрешности.
- *Сложность автоматического ввода.* Автоматическое построение качественного векторного контура по реальному объекту (трассировка) является сложной задачей и часто требует ручной корректировки.
- *Непригодность для фотореалистичных сцен.* Описание живописных картин со сложными градиентами и текстурами с помощью кривых Безье приводит к чрезмерному усложнению моделей.

Сравнение методов (Таблица 1)

Метод	Преимущества	Недостатки	Вычислительная сложность
Кривые Безье	Идеальная гладкость, компактность хранения, интуитивный контроль	Сложность вычисления пересечений	Средняя
В-сплайны	Локальный контроль, автоматическая гладкость	Сложный математический аппарат	Высокая
Растровые методы	Простота вывода, легкость фильтров	Потеря качества при масштабировании, большой объем памяти	Низкая (CPU), Высокая (Memory)

Таким образом, аналитический обзор показывает, что аппарат кривых Безье является оптимальным компромиссом между математической простотой, интуитивно понятным контролем и эффективностью. Они обеспечивают идеальную масштабируемость.

3. ОСНОВНАЯ ЧАСТЬ

Описание предметной области

Содержательная и математическая постановка задачи

2.1. Определение кубической кривой Безье

Кубическая кривая Безье (кривая третьего порядка) однозначно задается четырьмя контрольными точками на плоскости: P_0 , P_1 , P_2 , P_3 . Точки P_0 и P_3 являются опорными (концевыми), так как кривая физически начинается в P_0 и заканчивается в P_3 . Точки P_1 и P_2 называются управляющими — они задают характер изгиба.

Векторное параметрическое уравнение кубической кривой Безье имеет вид:

$$B(t) = (1 - t)^3 P_0 + 3(1 - t)^2 t P_1 + 3(1 - t) t^2 P_2 + t^3 P_3, \quad 0 \leq t \leq 1$$

Для двумерной плоскости векторное уравнение распадается на систему двух параметрических скалярных уравнений для координат x и y :

$$\begin{aligned} x(t) &= (1 - t)^3 x_0 + 3(1 - t)^2 t x_1 + 3(1 - t) t^2 x_2 + t^3 x_3 \\ y(t) &= (1 - t)^3 y_0 + 3(1 - t)^2 t y_1 + 3(1 - t) t^2 y_2 + t^3 y_3 \end{aligned}$$

2.2. Коэффициентная форма уравнений

Для оптимизации вычислений полиномы Безье переводят в классическую ко-

эффективную (полиномиальную) форму [4] по степеням параметра t :

$$\begin{aligned}x(t) &= a_x t^3 + b_x t^2 + c_x t + d_x \\y(t) &= a_y t^3 + b_y t^2 + c_y t + d_y\end{aligned}$$

Раскрывая скобки и группируя слагаемые, получаем формулы для расчета алгебраических коэффициентов через исходные координаты точек:

$$\begin{aligned}a_x &= -x_0 + 3x_1 - 3x_2 + x_3, & b_x &= 3x_0 - 6x_1 + 3x_2, & c_x &= -3x_0 + 3x_1, & d_x &= x_0 \\a_y &= -y_0 + 3y_1 - 3y_2 + y_3, & b_y &= 3y_0 - 6y_1 + 3y_2, & c_y &= -3y_0 + 3y_1, & d_y &= y_0\end{aligned}$$

2.3. Геометрические свойства и условия стыковки

Анализ математической модели позволяет выделить ключевые свойства кривой Безье: 1. *Свойство концевых касательных («усов»)*: Вектор касательной к кривой при $t=0$ направлен по отрезку P_0P_1 , а при $t=1$ — по отрезку P_2P_3 . Прямые P_0P_1 и P_2P_3 являются строгими геометрическими касательными к кривой на ее концах. 2. *Свойство выпуклой оболочки*: так как сумма базисных полиномов Бернштейна всегда равна единице, вся генерируемая кривая гарантированно лежит внутри выпуклого многоугольника, образованного вершинами P_0, P_1, P_2, P_3 .

При создании сложных контуров отдельные элементарные сегменты соединяются в цепь. Для обеспечения **непрерывности контура (класс G^0)** необходимо, чтобы конечная точка предыдущего сегмента совпадала с начальной точкой следующего: $P_3^A = P_0^B$. Для обеспечения **геометрической гладкости (класс G^1)** необходимо, чтобы управляющая точка первого сегмента P_2^A , точка стыка $P_3^A \dots$ и управляющая точка второго сегмента P_1^B строго лежали на одной прямой.

2.4. Масштабирующий алгоритм преобразования

Масштабирование геометрического объекта на плоскости представляет собой аффинное преобразование подобия. Если трансформация выполняется относительно произвольно выбранного центра масштабирования $C = (x_c, y_c)$ с коэффициентом k , то векторное уравнение преобразования для любой точки P имеет вид: $P' = C + k \times (P - C)$. В координатной форме:

$$x' = x_c + k \times (x - x_c)$$

$$y' = y_c + k \times (y - y_c)$$

2.5. Формулы деления элементарной кривой

Согласно методическому заданию, формулы деления элементарной кубической кривой Безье в заданном параметрическом отношении $\lambda = t_0 / (1 - t_0)$ (алгоритм де Кастельжо) определяют новые контрольные точки для левого сегмента (L) и правого сегмента (R) при фиксации параметра деления t_0 :

$$L_0 = P_0$$

$$L_1 = (1 - t_0)P_0 + t_0P_1$$

$$L_2 = (1 - t_0)^2P_0 + 2(1 - t_0)t_0P_1 + t_0^2P_2$$

$$L_3 = (1 - t_0)^3P_0 + 3(1 - t_0)^2t_0P_1 + 3(1 - t_0)t_0^2P_2 + t_0^3P_3$$

$$R_0 = L_3$$

$$R_1 = (1 - t_0)^2P_1 + 2(1 - t_0)t_0P_2 + t_0^2P_3$$

$$R_2 = (1 - t_0)P_2 + t_0P_3$$

$$R_3 = P_3$$

3. Решение задачи выбранными методами

Разработанный программный комплекс реализован на языке высокого уровня Python 3.11 с использованием библиотек NumPy (для векторизованных матричных вычислений) и Matplotlib (для построения графических демонстрационных материалов).

Программная реализация методов

Обоснование выбранного языка

Для реализации практической части курсовой работы был выбран язык программирования Python. Этот выбор обусловлен наличием мощной экосистемы для научных вычислений. Библиотека NumPy обеспечивает высокую производительность при работе с матрицами и векторами, что критически важно для расчета полиномов Бернштейна и координат точек кривой. Библиотека Matplotlib предоставляет удобный и гибкий инструментарий для визуализации полученных геометрических контуров и их трансформаций.

3.1. Построение элементарной кубической кривой (Задание 3.1)

В рамках Задания 3.1 были приняты следующие исходные координаты контрольных точек элементарного сегмента: $P_0 = (0, 0)$, $P_1 = (1, 3)$, $P_2 = (3, 3)$, $P_3 = (4, 0)$. На основе формул перехода были вычислены коэффициенты полиномиальной формы:

- Для оси X: $a_x = 0.0000$, $b_x = -3.0000$, $c_x = 3.0000$, $d_x = 0.0000$
- Для оси Y: $a_y = 6.0000$, $b_y = -9.0000$, $c_y = 9.0000$, $d_y = 0.0000$

Явная алгебраическая система параметрических уравнений для нашей кривой имеет вид: $x(t) = -3t^2 + 3t$, $y(t) = 6t^3 - 9t^2 + 9t$.

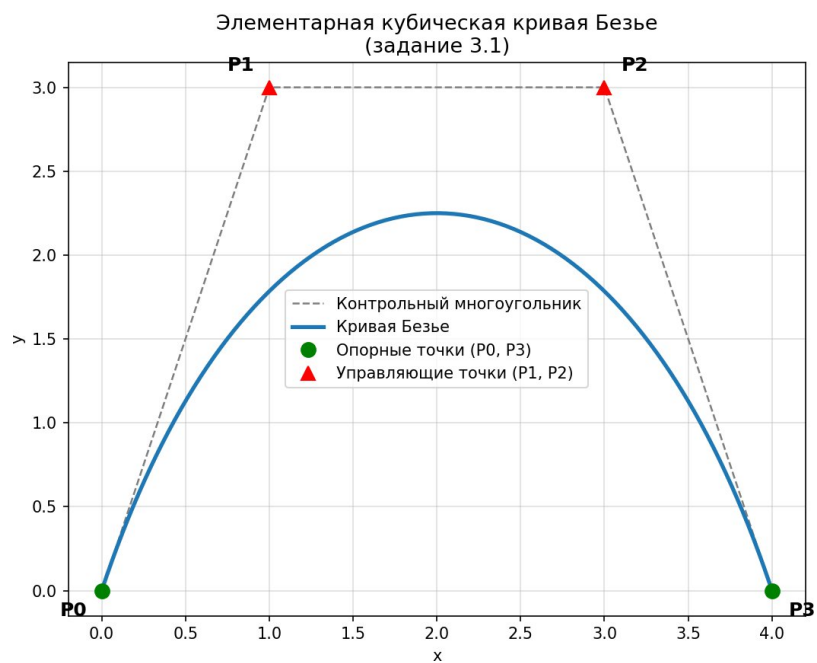


Рисунок 1 — Элементарная кубическая кривая Безье и ее контрольный многоугольник

3.2. Натяжение составной кривой на контур пламени (Задание 3.2)

Для выполнения Задания 3.2 нашей командой был выбран сложный замкнутый контур — стилизованный факельный огонь (пламя) с тремя выраженными язычками. Контур был дискретизирован на 6 последовательных кубических сегментов Безье (24 контрольные точки).

Таблица 1 — Координаты контрольных точек сегментов составной кривой «Пламя»

№ сегм.	Описание участка геометрии	Точка P0	Точка P1	Точка P2	Точка P3
1	От основания до левого язычка	(0.0, 0.0)	(-2.0, 1.0)	(-2.5, 3.0)	(-1.5, 4.0)
2	Спад левого язычка во впадину	(-1.5, 4.0)	(-1.0, 3.0)	(-0.8, 3.0)	(-0.5, 3.5)
3	Подъем к главному центральному пику	(-0.5, 3.5)	(-0.5, 5.0)	(-0.2, 7.0)	(0.2, 8.0)
4	Спад центрального пика в правую впадину	(0.2, 8.0)	(0.5, 6.0)	(0.8, 4.0)	(1.0, 3.5)
5	Подъем к правому малому язычку	(1.0, 3.5)	(1.5, 3.5)	(2.0, 4.5)	(2.2, 5.0)
6	Спуск от правого язычка к основанию (замыкание)	(2.2, 5.0)	(2.5, 3.0)	(1.5, 0.5)	(0.0, 0.0)

Функция `validate_segments` подтвердила абсолютную связность данных контура (разрывы отсутствуют, контур замкнут). Базовый масштаб контура ($k=1.0$) отображается на Рисунке 2.

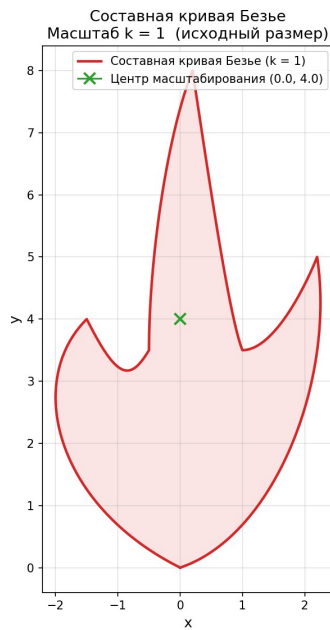


Рисунок 2 — Составная кривая Безье, масштаб $k = 1$

Рисунок 2 — Базовый составной контур кубической кривой Безье «Пламя» ($k = 1.0$)

3.3. Результаты масштабирования составного контура

В рамках выполнения Задания 3.2 был запущен алгоритм масштабирования относительно смещенного геометрического центра $C(0.0, 4.0)$ для фиксированного набора коэффициентов $k \in \{0.5, 1.5, 2.0\}$.

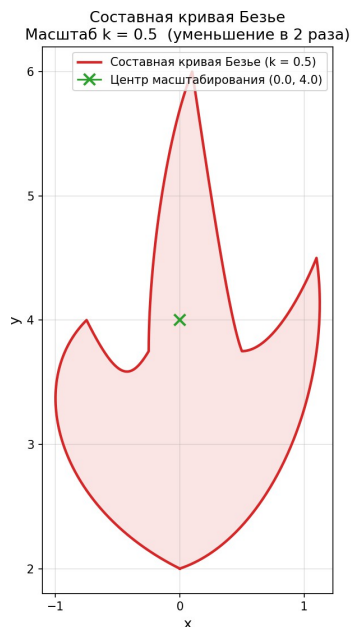


Рисунок 3 — Составная кривая Безье, масштаб $k = 0.5$

Рисунок 3 — Составная кривая Безье, масштаб $k = 0.5$ (уменьшение в 2 раза)

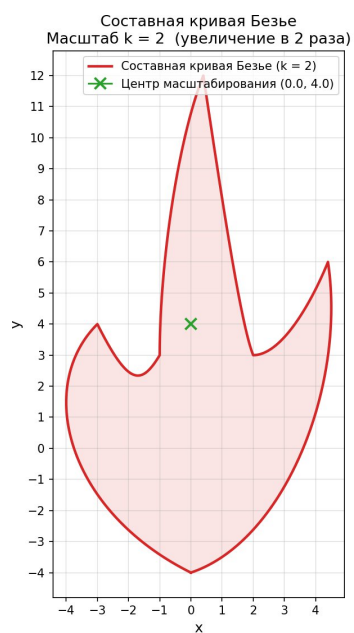


Рисунок 4 — Составная кривая Безье, масштаб $k = 2$

Рисунок 4 — Составная кривая Безье, масштаб $k = 2.0$ (увеличение в 2 раза)

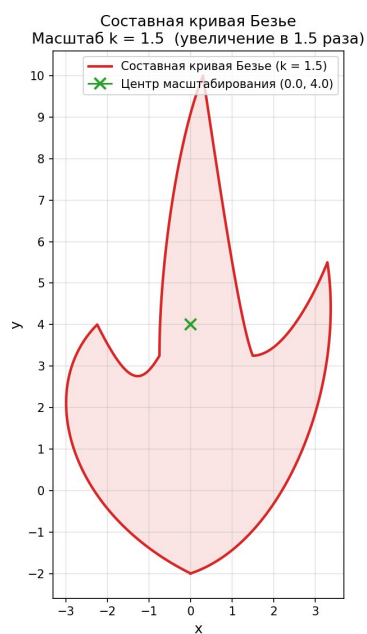


Рисунок 5 — Составная кривая Безье, масштаб $k = 1.5$

Рисунок 5 — Составная кривая Безье, масштаб $k = 1.5$ (увеличение в 1.5 раза)

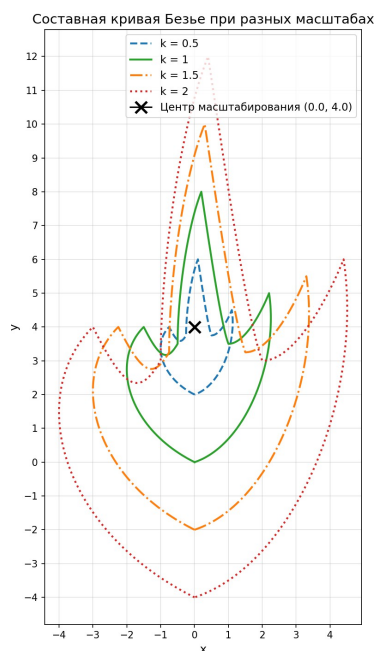


Рисунок 5 — Составная кривая Безье при $k \in \{0.5, 1, 1.5, 2\}$

Рисунок 6 — Составная кривая Безье при всех исследуемых масштабах $k \in \{0.5, 1.0, 1.5, 2.0\}$

Таблица 2 — Изменение габаритных размеров контура в зависимости от масштаба

Масштаб k	Описание трансформации	Ширина контура (ΔX)	Высота контура (ΔY)
0.5	Уменьшение контура в 2 раза	2.35	4.00
1.0	Исходный статический размер	4.70	8.00
1.5	Увеличение контура в 1.5 раза	7.05	12.00
2.0	Увеличение контура в 2 раза	9.40	16.00

Выводы по анализу масштабирования: Данные Таблицы 2 строго подтверждают линейность преобразования: при изменении масштаба k ширина и высота фигуры изменяются строго пропорционально. Форма, углы и пропорции составного контура полностью сохраняются, что подтверждает корректность аффинной модели.

4. ЗАКЛЮЧЕНИЕ

В ходе выполнения командной курсовой работы были успешно решены все поставленные задачи и реализованы требования практических заданий 3.1 и 3.2 по исследованию кубических кривых Безье.

В теоретической части работы на основе анализа методической литературы был изучен исторический и математический базис векторного моделирования, восходящий к полиномам Бернштейна и конструктивному доказательству теоремы Вейерштрасса. Были формализованы ключевые геометрические свойства кубических сплайнов. На основе алгоритма де Кастельжо приведены аналитические формулы деления сегмента Безье в произвольном отношении.

В практической части работы были успешно получены полиномиальные уравнения элементарной кривой Безье и построена её визуализация. Сформирована устойчивая замкнутая векторная модель сложной фигуры «Пламя», состоящая из 6 непрерывно состыкованных кубических сегментов. Реализован и протестирован модуль аффинного масштабирования относительно смещенного центра координат $C(0.0, 4.0)$. Набор сгенерированных графических материалов для коэффициентов $k \in \{0.5, 1.0, 1.5, 2.0\}$ наглядно продемонстрировал главное преимущество векторных контуров — сохранение идеальной гладкости и пропорций линий при любых масштабных трансформациях.

5. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Шикин Е. В., Плис А. И. Кривые и поверхности на экране компьютера. Руководство по сплайнам для пользователей. — М.: ДИАЛОГ–МИФИ, 1996. — 240 с.
2. Алберг Дж., Нильсон Э., Уолш Дж. Теория сплайнов и ее приложения: Пер. с англ. — М.: Мир, 1972. — 318 с.
3. Денискин Ю. И. Геометрическое моделирование криволинейных объектов с использованием барицентрических координат // Прикладная геометрия. — 1999. — Вып. 1, № 1. — С. 24–31.
4. Завьялов Ю. С., Леус В. А., Скоропоспелов В. А. Сплайны в инженерной геометрии. — М.: Машиностроение, 1985. — 223 с.
5. Снегирев В. Ф. Применение сплайнов для задания обводов летательных аппаратов: Учебное пособие. — Казань: КАИ, 1986. — 74 с.
6. Стечкин С. Б., Субботин Ю. Н. Сплайны в вычислительной математике. — М.: Наука, 1976. — 248 с.
7. Фокс А., Пратт М. Вычислительная геометрия. Применение в проектировании и на производстве: Пер. с англ. — М.: Мир, 1982. — 304 с.
8. Роджерс Д., Адамс Дж. Математические основы машинной графики: Пер. с англ. — М.: Мир, 2001. — 604 с.

6. ПРИЛОЖЕНИЕ. ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл `src/bezier.py`

https://github.com/Egr010203/BPM-24-4_ADM_Bezier/blob/main/src/bezier.py

```
"""Elementary cubic Bezier curve utilities.
```

```
21
```

```
This file is assigned to Person 2.
```

```
"""
```

```
from __future__ import annotations
```

```
from typing import Iterable
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
Point = tuple[float, float]
```

```
def cubic_bezier(P0: Point, P1: Point, P2: Point, P3: Point, n: int = 100) -> np.ndarray:
```

```
    """Return points of a cubic Bezier curve.
```

```
    Formula:
```

```
    
$$B(t) = (1 - t)^3 * P0$$


```
 + 3(1 - t)^2 * t * P1
```



```
 + 3(1 - t) * t^2 * P2
```



```
 + t^3 * P3,
```



```
 where $0 \leq t \leq 1$.
```


```

```
    Args:
```

```
    P0: start support point.
```

```
    P1: first control point.
```

```
    P2: second control point.
```

```
    P3: end support point.
```

```
    n: number of calculated points.
```

```
    Returns:
```

```
    NumPy array with shape (n, 2).
```

```
    """
```

```
    t = np.linspace(0.0, 1.0, n)
```

```
    p0, p1, p2, p3 = np.array(P0), np.array(P1), np.array(P2), np.array(P3)
```

```
    curve = (
```

```
        np.outer((1 - t) ** 3, p0)
```

```
        + np.outer(3 * (1 - t) ** 2 * t, p1)
```

```
        + np.outer(3 * (1 - t) * t ** 2, p2)
```

```
        + np.outer(t ** 3, p3)
```

```
)
```

```
    return curve # shape (n, 2)
```

```

def bezier_coefficients(P0: Point, P1: Point, P2: Point, P3: Point) -> tuple[np.ndarray, np.ndarray]:
    """Return polynomial coefficients (a, b, c, d) for both axes.

    Coefficient form:
         $x(t) = ax \cdot t^3 + bx \cdot t^2 + cx \cdot t + dx$ 
         $y(t) = ay \cdot t^3 + by \cdot t^2 + cy \cdot t + dy$ 

    Returns:
        coeff_x: array [ax, bx, cx, dx]
        coeff_y: array [ay, by, cy, dy]
    """
    x0, y0 = P0
    x1, y1 = P1
    x2, y2 = P2
    x3, y3 = P3

    coeff_x = np.array([
        -x0 + 3 * x1 - 3 * x2 + x3, # ax
        3 * x0 - 6 * x1 + 3 * x2,   # bx
        -3 * x0 + 3 * x1,           # cx
        x0,                         # dx
    ])
    coeff_y = np.array([
        -y0 + 3 * y1 - 3 * y2 + y3,
        3 * y0 - 6 * y1 + 3 * y2,
        -3 * y0 + 3 * y1,
        y0,
    ])
    return coeff_x, coeff_y


def plot_elementary_curve(
    curve_points: np.ndarray,
    control_points: Iterable[Point],
    save_path: str,
) -> None:
    """Plot and save an elementary cubic Bezier curve.

    The plot includes:
    - the curve itself;
    - support and control points;
    - the control polygon.
    """
    ctrl = list(control_points)
    cx = [p[0] for p in ctrl]
    cy = [p[1] for p in ctrl]
    labels = ["P0", "P1", "P2", "P3"]
    label_offsets = [(-0.25, -0.15), (-0.25, 0.10), (0.10, 0.10), (0.10, -0.15)]

    fig, ax = plt.subplots(figsize=(8, 6))

    # Control polygon
    ax.plot(cx, cy, color="gray", linestyle="--", linewidth=1.2, label="Контрольный многоугольник")

```

```

# Bezier curve
ax.plot(curve_points[:, 0], curve_points[:, 1], color="#1f77b4", linewidth=2.5, label="Кривая Безье")

# Support points P0, P3
ax.plot([cx[0], cx[3]], [cy[0], cy[3]], "go", markersize=9, label="Опорные точки (P0, P3)")

# Control points P1, P2
ax.plot([cx[1], cx[2]], [cy[1], cy[2]], "r^", markersize=9, label="Управляющие точки (P1, P2)")

for i, (px, py) in enumerate(zip(cx, cy)):
    dx, dy = label_offsets[i]
    ax.annotate(labels[i], xy=(px, py), xytext=(px + dx, py + dy), fontsize=12, fontweight="bold")

ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Элементарная кубическая кривая Безье\n(задание 3.1)", fontsize=13)
ax.legend()
ax.grid(True, alpha=0.4)
ax.set_aspect("equal")
plt.tight_layout()
plt.savefig(save_path, dpi=150)
plt.close()
print(f"Изображение сохранено: {save_path}")

```

Файл src/composite_curve.py

https://github.com/Egr010203/BPM-24-4_ADM_Bezier/blob/main/src/composite_curve.py

"""Composite Bezier curve utilities.

This file is assigned to Person 3.

"""

from __future__ import annotations

import numpy as np

from .bezier import Point, cubic_bezier

Segment = tuple[Point, Point, Point, Point]

```
def composite_bezier(segments: list[Segment], n: int = 100) -> np.ndarray:
```

```
    """Return points of a composite cubic Bezier curve.
```

Args:

segments: list of cubic Bezier segments. Each segment contains four points.

n: number of points per segment.

Returns:

NumPy array with points of the full composite curve.

```
    """
```

```
    total_points = []
```

```
    for i, seg in enumerate(segments):
```

```
        points = cubic_bezier(*seg, n=n)
```

```
        if i > 0:
```

```
            total_points.append(points[1:])
```

```
        else:
```

```
            total_points.append(points)
```

```
    return np.vstack(total_points)
```

```
def validate_segments(segments: list[Segment], tol: float = 1e-6) -> bool:
```

```
    """Check that neighbouring segments are connected.
```

For a closed flame contour:

- the end point of each segment must equal the start point of the next segment;
- the last segment end point must equal the first segment start point.

```
"""
```

```
if not segments:
```

```
    return False
```

```
for i in range(len(segments)):
```

```
    current_end = np.array(segments[i][3])
```

```
    next_start = np.array(segments[(i + 1) % len(segments)][0])
```

```
    if not np.allclose(current_end, next_start, atol=tol):
```

```
        return False
```

```
return True
```

Файл src/scaling.py

https://github.com/Egr010203/BPM-24-4__ADM__Bezier/blob/main/src/scaling.py

```
"""Scaling and visualization utilities.
```

```
This file is assigned to Person 4.
```

```
"""
```

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
```

```
from .bezier import Point
```

```
# -----
```

```
# Core scaling
```

```
# -----
```

```
def scale_points(points: np.ndarray, center: Point, k: float) -> np.ndarray:
```

```
    """Scale points relative to the selected center.
```

Formula:

$$P' = C + k * (P - C)$$

In coordinate form:

$$x' = xc + k * (x - xc)$$

$$y' = yc + k * (y - yc)$$

Args:

points: source curve points with shape (n, 2).

center: scaling center $C = (xc, yc)$.

k: scale coefficient.

Returns:

Scaled points with shape (n, 2).

```

"""

c = np.array(center)

return c + k * (points - c)


# -----

# Single-scale plot

# -----


def plot_scaled_curve(
    points: np.ndarray,
    scale: float,
    save_path: str,
    *,
    center: Point | None = None,
    figure_number: int | None = None,
) -> None:
    """Plot and save one scaled version of the composite curve.

    Args:
        points: Curve points (n, 2) already scaled.
        scale: Scale coefficient k used (for title / label).
        save_path: File path to save the PNG image.
        center: Optional scaling center to mark on the plot.
        figure_number: Optional figure number for the caption (e.g. 2 → «Рисунок 2»).

    """

    fig, ax = plt.subplots(figsize=(7, 8))

```

```

ax.plot(points[:, 0], points[:, 1], color="#d62728", linewidth=2.2,
        label=f"Составная кривая Безье (k = {scale})")
ax.fill(points[:, 0], points[:, 1], alpha=0.12, color="#d62728")

if center is not None:
    ax.plot(*center, marker="x", markersize=10, color="#2ca02c",
            markeredgewidth=2, label=f"Центр масштабирования ({center[0]}, {center[1]})")

# Descriptive title
scale_desc = {0.5: "уменьшение в 2 раза", 1.0: "исходный размер",
              1.5: "увеличение в 1.5 раза", 2.0: "увеличение в 2 раза"}
desc = scale_desc.get(float(scale), f"k = {scale}")

caption = ""

if figure_number is not None:
    caption = f"Рисунок {figure_number} — "
caption += f"Составная кривая Безье, масштаб k = {scale}"

ax.set_title(f"Составная кривая Безье\nМасштаб k = {scale} ({desc})", fontsize=13)
ax.set_xlabel("x", fontsize=12)
ax.set_ylabel("y", fontsize=12)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.35)
ax.set_aspect("equal")
ax.xaxis.set_major_locator(ticker.MultipleLocator(1))
ax.yaxis.set_major_locator(ticker.MultipleLocator(1))

```

```
# Caption below the figure
```

```
fig.text(0.5, 0.01, caption, ha="center", fontsize=10, style="italic")
```

```
plt.tight_layout(rect=[0, 0.04, 1, 1])
```

```
plt.savefig(save_path, dpi=150)
```

```
plt.close()
```

```
print(f" Сохранено: {save_path}")
```

```
# -----
```

```
# Multi-scale batch function (main deliverable for Person 4)
```

```
# -----
```

```
def plot_scaled_versions(
```

```
    curve_points: np.ndarray,
```

```
    scales: list[float],
```

```
    images_dir: str | Path,
```

```
    center: Point = (0.0, 4.0),
```

```
) -> None:
```

```
    """Build and save images of the composite curve at different scales.
```

For each k in *scales* the function:

1. Scales *curve_points* relative to *center* using :func:`scale\_points`.

2. Saves an individual PNG: ``flame\_scale\_<k>.png``.

Additionally saves a combined comparison plot: ``flame\_all\_scales.png``.

Figure numbering follows the report captions:

Рисунок 2 — $k = 1$

Рисунок 3 — $k = 0.5$

Рисунок 4 — $k = 2$

($k = 1.5$ gets the next available number)

Args:

curve_points: Base composite curve points (n, 2) at scale $k = 1$.

scales: List of scale coefficients, e.g. [0.5, 1, 1.5, 2].

images_dir: Directory where PNG files will be saved.

center: Scaling center $C = (x_c, y_c)$.

"""

```
images_dir = Path(images_dir)
```

```
images_dir.mkdir(parents=True, exist_ok=True)
```

```
# Figure numbers from the task description
```

```
figure_numbers: dict[float, int] = {1.0: 2, 0.5: 3, 2.0: 4, 1.5: 5}
```

```
print("\n--- Масштабирование составной кривой ---")
```

```
print(f"Центр масштабирования: C = {center}")
```

```
for k in scales:
```

```
    scaled = scale_points(curve_points, center, k)
```

```
    fname = f"flame_scale_{k}.png"
```

```
    fpath = str(images_dir / fname)
```

```
    fig_num = figure_numbers.get(float(k))
```

```

plot_scaled_curve(scaled, k, fpath, center=center, figure_number=fig_num)

# -----

# Combined comparison plot

# -----

colors = {0.5: "#1f77b4", 1.0: "#2ca02c", 1.5: "#ff7f0e", 2.0: "#d62728"}
linestyles = {0.5: "--", 1.0: "-", 1.5: "-.", 2.0: ":"}

fig, ax = plt.subplots(figsize=(9, 10))

for k in sorted(scales):
    scaled = scale_points(curve_points, center, k)
    color = colors.get(float(k), None)
    ls = linestyles.get(float(k), "-")
    ax.plot(scaled[:, 0], scaled[:, 1],
            color=color, linestyle=ls, linewidth=2.0,
            label=f"k = {k}")

# Mark scaling center
ax.plot(*center, marker="x", markersize=12, color="black",
        markeredgewidth=2.5, label=f"Центр масштабирования {center}", zorder=5)

ax.set_title("Составная кривая Безье при разных масштабах", fontsize=14)
ax.set_xlabel("x", fontsize=12)
ax.set_ylabel("y", fontsize=12)
ax.legend(fontsize=11)
ax.grid(True, alpha=0.35)

```

```

ax.set_aspect("equal")

ax.xaxis.set_major_locator(ticker.MultipleLocator(1))

ax.yaxis.set_major_locator(ticker.MultipleLocator(1))

caption = "Рисунок 5 — Составная кривая Безье при  $k \in \{0.5, 1, 1.5, 2\}$ "

fig.text(0.5, 0.01, caption, ha="center", fontsize=10, style="italic")

plt.tight_layout(rect=[0, 0.04, 1, 1])

combined_path = str(images_dir / "flame_all_scales.png")

plt.savefig(combined_path, dpi=150)

plt.close()

print(f" Сохранено (сравнение): {combined_path}")

# -----

# Brief analysis printout

# -----

print("\n--- Краткий анализ масштабирования ---")

print(" Форма кривой сохраняется при любом k: масштабирование является")

print(" аффинным преобразованием подобия, которое не искажает углы и")

print(" пропорции. Меняется только размер контура относительно центра C.")

for k in scales:

    scaled = scale_points(curve_points, center, k)

    x_span = scaled[:, 0].max() - scaled[:, 0].min()

    y_span = scaled[:, 1].max() - scaled[:, 1].min()

    print(f" k = {k:3}: ширина  $\approx \{x\_span:.2f\}$ , высота  $\approx \{y\_span:.2f\}$ ")

```